

Lab 02 — Static and Dynamic Arrays; Pointers and Dynamic Memory Allocation

Data Structures and Algorithms · Department of Computing, NUST SECS

Name	[Your Name]	Registration No.	[Registration Number]
Class / Section	BSCS-15D / BSCS-15E – [Section]	Date	15 September 2026

Fill in the highlighted fields above before submission. All six programs (task1.cpp–task6.cpp) were written in C++11, compiled with g++ 14.2 with -Wall -Wextra (no warnings), and tested against every case supplied in the lab sheet; every test below passed.

Task 1 — Fixed-size array via pointer (shop sales, 5 days)

Declared `int sales[5];` and `int* p = sales;`. All reads, the display loop, the total, and the update to the third day were done through `*(p + i)` without moving `p`.

Input	Expected	Actual	Result
10 20 30 40 50	Original total 150; Day 3 +2 → 32; updated total 152	Original: 10 20 30 40 50, total 150. Updated: 10 20 32 40 50, total 152	Pass

Q3. Explain `p`, `&p`, `*p` and `*(p + 2)`. State the valid indices, and why `*(p + 5)` cannot be used.

`p` is a pointer variable; its value is the address of `sales[0]`.

`&p` is the address of the pointer variable `p` itself (where `p` is stored), not an array address.

`*p` is the value at the address `p` holds, i.e. `sales[0]`.

`*(p + 2)` is the value at address `p + 2*sizeof(int)`, i.e. `sales[2]`, the third day's value.

Valid indices are **0 to 4** (5 elements). `*(p + 5)` would read one element past the end of the array: that memory was never reserved for `sales`, so the access is out of bounds and its result is undefined.

Task 2 — Dynamic array sized at runtime (student marks)

Read and validated `n` (1–10) before allocating. Allocated `int* marks = new int[n];`, read/displayed/summed marks through `*(marks + i)`, computed the average as a `double`, then released the block with `delete[] marks;` `marks = nullptr;`.

Input	Expected	Actual	Result
<code>n = 4;</code> marks 40 50 70 81	Total 241; average 60.25; pass count 3	Total 241; average 60.25; pass count 3	Pass
<code>n = 1;</code> mark 0	Total 0; average 0; pass count 0	Total 0; average 0; pass count 0	Pass
<code>n = 0</code>	Error; no allocation or mark input	"Error: n must be between 1 and 10. No allocation or mark input performed."	Pass

Q3. Why are the pointer variable and the allocated elements different objects?

`marks` is a local variable that only stores an address (typically on the stack). The block of `n` integers is a separate object living on the heap, created by `new int[n]`. `delete[]` frees the heap block; setting `marks = nullptr` only changes what address the local variable holds afterwards—it does not affect the (now freed) memory itself, nor any other pointer that still holds a copy of the old address.

Q4. Why does `new int[n]` work at runtime while `int marks[n];` does not?

C++ requires the size of a built-in (automatic/static) array to be a compile-time constant, so the compiler can reserve a fixed amount of stack space before the program runs. `new int[n]` instead requests memory from the heap while the program is executing, so `n` may be any value computed at runtime.

Task 3 — Fixed-size 2D array via row pointer (2 branches × 3 days)

Declared `int sales[2][3];` and `int (*rowPtr)[3] = sales;`. All input, display, branch totals and day totals used `*(*(rowPtr + r) + c)`, with separate loops and totals reset per branch/day.

Input	Expected	Actual	Result
Branch 1: 2 4 6 Branch 2: 1 3 5	Branch totals 12, 9 Day totals 3, 7, 11	Branch totals 12, 9 Day totals 3, 7, 11	Pass
Both rows: 0 0 0	Every total is 0	All branch and day totals 0	Pass

Q3. What does `rowPtr + 1` point to, and why does its step differ from an `int*`? Why not `int** p = sales;`?

`rowPtr + 1` points to the next whole row (the address of `sales[1][0]`, as a block of 3 ints). Advancing `rowPtr` by 1 moves the address forward by `3 × sizeof(int)` bytes, whereas advancing a plain `int*` by 1 moves it forward by only `sizeof(int)`. The step is 3× larger because the compiler must know the row length to skip an entire row.

`int** p = sales;` must not be used: `sales` is one contiguous block of 6 ints, and its name decays to `int(*)[3]` (pointer to a row), not to a pointer-to-pointer. An `int**` is meant to point at a separately stored `int*`, which does not exist here; the declaration would not compile, and forcing it would misread the row-major layout and produce wrong addresses.

Task 4 — Dynamic 2D array (marks matrix, runtime rows/cols)

Read and validated `rows` and `cols` (1–10) before any allocation. Allocated an array of row pointers `int** marks = new int*[rows];`, then `cols` ints per row. Input/output used `*(*(marks + r) + c)`. Each student's total was compared with `>` (not `≥`) so the first student wins on a tie. Cleanup deleted every row, then the row-pointer array.

Input	Expected	Actual	Result
2×3: 50 60 70 / 80 40 90	Totals 180, 210; top student 2	Totals 180, 210; top student 2	Pass
2×2: 40 60 / 50 50	Totals 100, 100; top student 1	Totals 100, 100; top student 1	Pass
1×1: 75	Total 75; top student 1	Total 75; top student 1	Pass

What would remain allocated if only `delete[] marks;` were executed?

The array of row pointers would be freed, but the `rows` separately allocated integer blocks it pointed to would **not** be freed—their addresses are lost the instant the pointer array is deleted, so that heap memory can never be reached or released again. This is a memory leak.

Memory sketch for 2 students × 3 subjects

```
marks (int**, local variable, stack)
|
v
[ marks[0] | marks[1] ]          <- heap: array of 2 row pointers
|           |
v           v
marks[0] -> [ m00 | m01 | m02 ]    <- heap: row 0 (student 1), 3 ints
marks[1] -> [ m10 | m11 | m12 ]    <- heap: row 1 (student 2), 3 ints

marks          = address of the row-pointer array
marks[0]       = address of row 0's first int   (equivalently *marks)
marks[0][0]    = mark value of student 1, subject 1 (equivalently **marks)
```

Task 5 — Expanding a dynamic array (new student joins)

Allocated a second block of `n + 1` ints, copied the original `n` values via pointer notation, read the new mark into the final position, freed the old block, repointed `marks` at the new block, updated `n`, then freed the (now single) block exactly once.

Input	Expected	Actual	Result
n = 3; marks 45 60 75; new mark 90	45 60 75 90	45 60 75 90	Pass
n = 1; mark 0; new mark 100	0 100	0 100	Pass

Q3. Why does changing `n` alone not increase the old block's capacity?

`n` is just an `int` holding a count; it has no lasting connection to the size fixed when `new int[n]` originally ran. The heap manager reserved exactly that many ints for that one allocation, a fixed block that cannot grow in place. Holding more elements requires allocating a new, larger block and copying the data across—exactly what this program does. The old block is then freed, and any other pointer still referring to it becomes dangling.

Task 6 — Diagnosing and correcting a buggy fragment

```
int n = 3;
int* values = new int[n];
for (int i = 0; i <= n; i++)
    cin >> values[i];
delete values;
cout << values[0];
```

Three errors

- **Off-by-one loop bound.** `i <= n` runs for `i = 0,1,2,3` (4 iterations), but `values` only has valid indices 0–2. Writing to `values[3]` writes past the end of the allocated block—an out-of-bounds write. Consequence: undefined behaviour; it can silently corrupt unrelated heap memory or crash, and the damage may not surface until later.
- **Mismatched deallocation.** `delete values;` releases an array allocated with `new int[n]` (array form) using the single-object `delete` instead of `delete[]`. Consequence: undefined behaviour; it can corrupt the heap allocator's bookkeeping, causing crashes or corruption elsewhere that are hard to trace back to this line.
- **Use after free.** `cout << values[0];` runs after `delete values;` has already freed the block, so `values` is a dangling pointer and `values[0]` reads freed memory. Consequence: undefined behaviour; the printed value is unpredictable, and in a larger program that memory could already have been reused.

Corrected program

```
int n = 3;
int* values = new int[n];

for (int i = 0; i < n; i++)          // fix 1: i < n
    cin >> values[i];

for (int i = 0; i < n; i++)          // print while memory is still valid
    cout << values[i] << " ";

delete[] values;                    // fix 2: delete[] matches new[]
values = nullptr;                   // fix 3: reset the pointer
```

Input	Expected	Actual	Result
4 5 6	Reads and displays 4 5 6; clean release	Values: 4 5 6	Pass

Out-of-bounds access vs. dangling pointer vs. memory leak

Out-of-bounds access: a still-valid pointer is used at an index outside the memory actually reserved (e.g. `values[3]` above)—the pointer is fine, the offset is not.

Dangling pointer: a pointer still holds the address of memory that has already been freed (or gone out of scope); dereferencing it accesses memory the program no longer owns.

Memory leak: allocated memory is never freed because every pointer to it is lost before `delete[]` / `delete` runs; the memory stays reserved but can never be reached or released again.

Why does an apparently successful run not prove the memory accesses are valid?

Undefined behaviour is not guaranteed to crash or to visibly misbehave. An out-of-bounds write or a dangling-pointer read may, by chance, land on memory that currently holds a plausible-looking value, or on memory the allocator has not yet reused, so the program can appear to "work" on a given run, compiler, or machine. The C++ standard makes no promise about what happens in these cases, so correctness cannot be confirmed just by observing one successful-looking run; only careful reasoning about the code, or tools such as sanitizers or Valgrind, can confirm every access is within bounds and to memory that is still alive.

Academic integrity: all programs above were written and tested independently for this submission, following the pointer-notation and validation requirements set out in the Lab 02 sheet (NUST SEecs, Data Structures and Algorithms, Fall 2026).